# PIXELCRAFT STUDIO PLUS DEVELOPMENT USING PYTHON

## AIM

To design and develop an interactive 2D graphic design editor named PixelCraft Studio Plus using Python Tkinter that allows users to create designs with text, rectangles, ellipses, lines and images, edit fill colour, font and stroke properties, manage layers, apply ready-made templates, undo and redo changes, zoom and show a grid, save and open projects as JSON files, and export the final design.

## PROCEDURE

1. Start the program and import the required Python libraries: tkinter (with ttk, colorchooser, filedialog, messagebox and simpledialog), json, os and copy.
2. Define the theme colour constants (BG, PANEL, CARD, ACCENT, TEXT, MUTED) and the fill colour PALETTE used throughout the interface.
3. Create the main application window using Tkinter, set the title as "PixelCraft Studio Plus" and set the window size and minimum size.
4. Create the PixelCraft class and initialise the application state: canvas size (800 x 520), object list, selected object, current tool, colour, font settings, undo/redo history, zoom level and grid option.
5. Design the user interface using Frame, Label, Button and Canvas widgets: a header bar (New, Open, Save, Undo, Redo, Export), a left Design Tools panel, a central design canvas, a right Properties panel and a bottom status bar.
6. Implement the drawing tools (Select / Move, Text, Rectangle, Ellipse and Line) using the mouse events ButtonPress-1, B1-Motion and ButtonRelease-1 to create, select and drag objects.
7. Add the Quick Insert options: Add Heading, Add Subtitle, Add Image, Duplicate (Ctrl+D) and Delete Selected.
8. Create the Properties panel with the fill colour palette, Custom Color chooser, font family, font size, Bold and Italic options, stroke width and the Apply Properties button.
9. Implement the Layers list with Bring to Front, Send to Back and Hide/Show options, and refresh it whenever the design changes.
10. Write the redraw() function to draw every object on the Canvas, show the selection box and optional grid, and add the Zoom + , Zoom - and Grid controls.
11. Implement Undo and Redo using snapshot history, and bind keyboard shortcuts (Ctrl+S, Ctrl+O, Ctrl+N, Ctrl+Z, Ctrl+Y, Ctrl+D and Delete).
12. Create five ready-made templates: Instagram Post, Story / Announcement, YouTube Thumbnail, Event Poster and Quote Card.

13. Provide Save and Open options using JSON files, and an Export option that saves the canvas as a PostScript file, with error handling using messagebox.
14. Execute the program and verify the tools, text and shape creation, property changes, layers, templates, undo/redo, save/open and export functions.

## PROGRAM CODE

```
import tkinter as tk
from tkinter import ttk, colorchooser, filedialog, messagebox, simpledialog
import json
import os
import copy

# PixelCraft Studio Plus
# Python standard library + Tkinter. No pip installation required.

BG = "#171923"
PANEL = "#202330"
CARD = "#292D3E"
ACCENT = "#7C5CFC"
TEXT = "#F1F2F8"
MUTED = "#A4A9BC"
PALETTE = ["#7C5CFC", "#FF6584", "#F7B267", "#52D6A6",
           "#58A6FF", "#F1F2F8", "#222222", "#FFFFFF"]


class PixelCraft:
    def __init__(self, root):
        self.root = root
        self.root.title("PixelCraft Studio Plus")
        self.root.geometry("1400x850")
        self.root.minsize(1100, 700)
        self.root.configure(bg=BG)

        self.canvas_w, self.canvas_h = 800, 520
        self.bg_color = "#FFFFFF"
        self.objects = []
        self.selected_id = None
        self.next_id = 1
        self.current_tool = "select"
        self.current_color = ACCENT
        self.stroke_color = ACCENT
        self.stroke_width = 3
        self.font_family = "Arial"
        self.font_size = 32
        self.font_bold = True
        self.font_italic = False
        self.drag_start = None
        self.drag_item_id = None
        self.temp_obj = None
        self.history = []
        self.future = []
        self.photo_refs = {}
        self.zoom = 1.0
        self.project_path = None
        self.project_name = "Untitled Design"
        self.show_grid = False
        self.grid_ids = []

        self.setup_ui()
        self.snapshot()
        self.redraw()
        self.refresh_layers()
        self.status("Ready — create something great!")

    def button(self, parent, text, command, bg=CARD, fg=TEXT, **kwargs):
        return tk.Button(
            parent, text=text, command=command, bg=bg, fg=fg,
            activebackground=ACCENT, activeforeground="white",
            relief="flat", bd=0, cursor="hand2",
            font=("Segoe UI", 9), padx=9, pady=7, **kwargs
        )

    def label(self, parent, text, size=10, color=TEXT, bold=False):
```

```
        return tk.Label(parent, text=text, bg=parent.cget("bg"), fg=color,
                        font=("Segoe UI", size, "bold" if bold else "normal"))

    def setup_ui(self):
        header = tk.Frame(self.root, bg=PANEL, height=60)
        header.pack(fill="x")
        header.pack_propagate(False)
        tk.Label(header, text="✦", bg=PANEL, fg=ACCENT,
                 font=("Segoe UI", 25, "bold")).pack(side="left", padx=(18, 7))
        tk.Label(header, text="PIXELCRAFT", bg=PANEL, fg=TEXT,
                 font=("Segoe UI", 15, "bold")).pack(side="left")
        tk.Label(header, text=" STUDIO PLUS", bg=PANEL, fg=MUTED,
                 font=("Segoe UI", 9)).pack(side="left", padx=6, pady=(6, 0))

        for text, cmd, color in [
            ("□ New", self.new_project, CARD),
            ("Open", self.open_project, CARD),
            ("Save", self.save_project, CARD),
            ("Undo", self.undo, CARD),
            ("Redo", self.redo, CARD),
            ("Export", self.export_design, ACCENT),
        ]:
            self.button(header, text, cmd, bg=color).pack(side="right", padx=4, pady=12)

        body = tk.Frame(self.root, bg=BG)
        body.pack(fill="both", expand=True, padx=10, pady=10)

        self.left = tk.Frame(body, bg=PANEL, width=175)
        self.left.pack(side="left", fill="y", padx=(0, 8))
        self.left.pack_propagate(False)
        self.build_left_panel()

        center = tk.Frame(body, bg=BG)
        center.pack(side="left", fill="both", expand=True)

        top = tk.Frame(center, bg=BG)
        top.pack(fill="x", pady=(0, 8))
        self.title_label = tk.Label(top, text=self.project_name, bg=BG, fg=TEXT,
                                    font=("Segoe UI", 13, "bold"))
        self.title_label.pack(side="left")
        tk.Label(top, text="  •  800 × 520 px", bg=BG, fg=MUTED,
                 font=("Segoe UI", 9)).pack(side="left")
        self.button(top, "Grid", self.toggle_grid).pack(side="right")
        self.button(top, "Zoom −", lambda: self.set_zoom(max(.5, self.zoom - .1))).pack(side="right",
                padx=3)
        self.button(top, "Zoom +", lambda: self.set_zoom(min(1.5, self.zoom + .1))).pack(side="right",
                padx=3)

        self.canvas_wrap = tk.Frame(center, bg=CARD, padx=12, pady=12)
        self.canvas_wrap.pack(fill="both", expand=True)
        self.canvas = tk.Canvas(self.canvas_wrap, width=self.canvas_w, height=self.canvas_h,
                                bg=self.bg_color, highlightthickness=0, cursor="arrow")
        self.canvas.pack(expand=True)
        self.canvas.bind("<ButtonPress-1>", self.on_press)
        self.canvas.bind("<B1-Motion>", self.on_motion)
        self.canvas.bind("<ButtonRelease-1>", self.on_release)

        self.right = tk.Frame(body, bg=PANEL, width=230)
        self.right.pack(side="right", fill="y", padx=(8, 0))
        self.right.pack_propagate(False)
        self.build_right_panel()

        footer = tk.Frame(self.root, bg=PANEL, height=28)
        footer.pack(fill="x", side="bottom")
        footer.pack_propagate(False)
        self.status_label = tk.Label(footer, text="Ready", bg=PANEL, fg=MUTED,
                                     font=("Segoe UI", 9), anchor="w")
        self.status_label.pack(side="left", padx=12)
        tk.Label(footer, text="PYTHON • TKINTER", bg=PANEL, fg=MUTED,
                 font=("Segoe UI", 8)).pack(side="right", padx=12)

        self.root.bind("<Delete>", lambda e: self.delete_selected())
        self.root.bind("<Control-s>", lambda e: self.save_project())
        self.root.bind("<Control-o>", lambda e: self.open_project())
        self.root.bind("<Control-n>", lambda e: self.new_project())
        self.root.bind("<Control-z>", lambda e: self.undo())
        self.root.bind("<Control-y>", lambda e: self.redo())
        self.root.bind("<Control-d>", lambda e: self.duplicate_selected())
```

```
def build_left_panel(self):
    self.label(self.left, "DESIGN TOOLS", 10, MUTED, True).pack(anchor="w", padx=14, pady=(17, 9))
    self.tool_buttons = {}
    tool_list = [
        ("↖", "Select / Move", "select"),
        ("T", "Text", "text"),
        ("▭", "Rectangle", "rect"),
        ("○", "Ellipse", "oval"),
        ("╱", "Line", "line"),
    ]
    for symbol, name, key in tool_list:
        b = self.button(self.left, f"{symbol}   {name}",
                        lambda k=key: self.set_tool(k),
                        bg=ACCENT if key == "select" else PANEL,
                        anchor="w")
        b.pack(fill="x", padx=8, pady=2)
        self.tool_buttons[key] = b

    tk.Frame(self.left, bg=CARD, height=1).pack(fill="x", padx=12, pady=12)
    self.label(self.left, "QUICK INSERT", 9, MUTED, True).pack(anchor="w", padx=14, pady=(0, 7))
    self.button(self.left, "□ Add Heading", self.add_heading).pack(fill="x", padx=8, pady=2)
    self.button(self.left, "□ Add Subtitle", self.add_subtitle).pack(fill="x", padx=8, pady=2)
    self.button(self.left, "▧ Add Image", self.add_image).pack(fill="x", padx=8, pady=2)
    self.button(self.left, "Duplicate  Ctrl+D", self.duplicate_selected).pack(fill="x", padx=8,
            pady=2)
    self.button(self.left, "Delete Selected", self.delete_selected, bg="#49303B").pack(fill="x",
            padx=8, pady=2)

    tk.Frame(self.left, bg=CARD, height=1).pack(fill="x", padx=12, pady=12)
    self.label(self.left, "TEMPLATES", 9, MUTED, True).pack(anchor="w", padx=14, pady=(0, 7))
    for name, key in [
        ("Instagram Post", "instagram"),
        ("Story / Announcement", "story"),
        ("YouTube Thumbnail", "youtube"),
        ("Event Poster", "event"),
        ("Quote Card", "quote"),
    ]:
        self.button(self.left, name, lambda k=key: self.load_template(k)).pack(fill="x", padx=8,
                pady=2)

    tk.Frame(self.left, bg=CARD, height=1).pack(fill="x", padx=12, pady=12)
    self.button(self.left, "Canvas Background", self.change_background).pack(fill="x", padx=8,
            pady=2)

def build_right_panel(self):
    self.label(self.right, "PROPERTIES", 11, TEXT, True).pack(anchor="w", padx=14, pady=(17, 4))
    self.label(self.right, "Select an object to edit", 9, MUTED).pack(anchor="w", padx=14, pady=(0,
            12))
    tk.Frame(self.right, bg=CARD, height=1).pack(fill="x", padx=12)

    self.label(self.right, "FILL COLOR", 9, MUTED, True).pack(anchor="w", padx=14, pady=(13, 6))
    palette = tk.Frame(self.right, bg=PANEL)
    palette.pack(fill="x", padx=12)
    for i, color in enumerate(PALETTE):
        tk.Button(palette, bg=color, width=3, height=1, relief="flat",
                  command=lambda c=color: self.set_color(c)).grid(row=i // 4, column=i % 4, padx=3,
                          pady=3)
    self.button(self.right, "Custom Color…", self.choose_color).pack(fill="x", padx=12, pady=6)

    tk.Frame(self.right, bg=CARD, height=1).pack(fill="x", padx=12, pady=10)
    self.label(self.right, "TEXT SETTINGS", 9, MUTED, True).pack(anchor="w", padx=14, pady=(0, 6))

    self.label(self.right, "Font family", 9, MUTED).pack(anchor="w", padx=14)
    self.font_family_var = tk.StringVar(value="Arial")
    ttk.Combobox(self.right, textvariable=self.font_family_var,
                 values=["Arial", "Times New Roman", "Courier New", "Verdana", "Georgia", "Comic
                         Sans MS"],
                 state="readonly").pack(fill="x", padx=12, pady=(3, 8))

    self.label(self.right, "Font size", 9, MUTED).pack(anchor="w", padx=14)
    self.font_size_var = tk.StringVar(value="32")
    ttk.Combobox(self.right, textvariable=self.font_size_var,
                 values=["12", "16", "20", "24", "28", "32", "40", "48", "60", "72"],
                 state="readonly").pack(fill="x", padx=12, pady=(3, 8))

    self.bold_var = tk.BooleanVar(value=True)
    self.italic_var = tk.BooleanVar(value=False)
    tk.Checkbutton(self.right, text="Bold", variable=self.bold_var, bg=PANEL, fg=TEXT,
                   selectcolor=CARD, activebackground=PANEL).pack(anchor="w", padx=12)
```

```
        tk.Checkbutton(self.right, text="Italic", variable=self.italic_var, bg=PANEL, fg=TEXT,
                       selectcolor=CARD, activebackground=PANEL).pack(anchor="w", padx=12)

        self.label(self.right, "Stroke width", 9, MUTED).pack(anchor="w", padx=14, pady=(8, 0))
        self.stroke_var = tk.StringVar(value="3")
        ttk.Combobox(self.right, textvariable=self.stroke_var,
                     values=["1", "2", "3", "4", "6", "8", "12"],
                     state="readonly").pack(fill="x", padx=12, pady=(3, 8))
        self.button(self.right, "Apply Properties", self.apply_properties, bg=ACCENT).pack(fill="x",
                padx=12, pady=5)

        tk.Frame(self.right, bg=CARD, height=1).pack(fill="x", padx=12, pady=10)
        self.label(self.right, "LAYERS", 10, TEXT, True).pack(anchor="w", padx=14, pady=(0, 6))
        layer_frame = tk.Frame(self.right, bg=PANEL)
        layer_frame.pack(fill="both", expand=True, padx=10)
        self.layer_list = tk.Listbox(layer_frame, bg=CARD, fg=TEXT, selectbackground=ACCENT,
                                     relief="flat", bd=0, font=("Segoe UI", 9),
                                     exportselection=False, height=8)
        self.layer_list.pack(side="left", fill="both", expand=True)
        scroll = ttk.Scrollbar(layer_frame, orient="vertical", command=self.layer_list.yview)
        scroll.pack(side="right", fill="y")
        self.layer_list.configure(yscrollcommand=scroll.set)
        self.layer_list.bind("<<ListboxSelect>>", self.on_layer_select)

        layer_actions = tk.Frame(self.right, bg=PANEL)
        layer_actions.pack(fill="x", padx=10, pady=6)
        self.button(layer_actions, "↑ Front", self.bring_front).pack(side="left", expand=True,
                fill="x", padx=2)
        self.button(layer_actions, "↓ Back", self.send_back).pack(side="left", expand=True, fill="x",
                padx=2)
        self.button(layer_actions, "Hide/Show", self.toggle_visibility).pack(side="left", expand=True,
                fill="x", padx=2)

    def status(self, text):
        self.status_label.configure(text="●  " + text)

    def set_tool(self, tool):
        self.current_tool = tool
        for key, b in self.tool_buttons.items():
            b.configure(bg=ACCENT if key == tool else PANEL)
        self.canvas.configure(cursor="arrow" if tool == "select" else "crosshair")
        self.status("Tool: " + tool.title())

    def new_object(self, kind, x1, y1, x2=None, y2=None, **kwargs):
        obj = {
            "id": self.next_id,
            "type": kind,
            "x1": x1, "y1": y1,
            "x2": x1 if x2 is None else x2,
            "y2": y1 if y2 is None else y2,
            "fill": kwargs.get("fill", self.current_color),
            "outline": kwargs.get("outline", self.stroke_color),
            "width": int(kwargs.get("width", self.stroke_width)),
            "text": kwargs.get("text", ""),
            "font": kwargs.get("font", self.font_family),
            "size": int(kwargs.get("size", self.font_size)),
            "bold": bool(kwargs.get("bold", self.font_bold)),
            "italic": bool(kwargs.get("italic", self.font_italic)),
            "anchor": kwargs.get("anchor", "nw"),
            "path": kwargs.get("path", ""),
            "visible": True
        }
        self.next_id += 1
        self.objects.append(obj)
        return obj["id"]

    def find_obj(self, obj_id):
        for obj in self.objects:
            if obj["id"] == obj_id:
                return obj
        return None

    def redraw(self):
        self.canvas.delete("all")
        self.photo_refs.clear()
        self.canvas.configure(bg=self.bg_color)
        self.grid_ids = []
        if self.show_grid:
            for x in range(0, self.canvas_w, 25):
```

```
            self.grid_ids.append(self.canvas.create_line(x, 0, x, self.canvas_h,
                                                         fill="#E6E6E6", dash=(1, 5), tags="grid"))
        for y in range(0, self.canvas_h, 25):
            self.grid_ids.append(self.canvas.create_line(0, y, self.canvas_w, y,
                                                         fill="#E6E6E6", dash=(1, 5), tags="grid"))
        self.canvas.tag_lower("grid")

    for obj in self.objects:
        if not obj.get("visible", True):
            continue
        x1, y1, x2, y2 = obj["x1"], obj["y1"], obj["x2"], obj["y2"]
        kind = obj["type"]
        fill = obj.get("fill", self.current_color)
        outline = obj.get("outline", fill)
        width = int(obj.get("width", 1))
        if kind == "rect":
            item = self.canvas.create_rectangle(x1, y1, x2, y2, fill=fill, outline=outline,
                    width=width)
        elif kind == "oval":
            item = self.canvas.create_oval(x1, y1, x2, y2, fill=fill, outline=outline, width=width)
        elif kind == "line":
            item = self.canvas.create_line(x1, y1, x2, y2, fill=fill, width=width)
        elif kind == "text":
            styles = []
            if obj.get("bold"):
                styles.append("bold")
            if obj.get("italic"):
                styles.append("italic")
            font_tuple = (obj.get("font", "Arial"), int(obj.get("size", 32)), *styles)
            item = self.canvas.create_text(x1, y1, text=obj.get("text", ""),
                                           fill=fill, font=font_tuple, anchor=obj.get("anchor",
                                                   "nw"))
        elif kind == "image":
            try:
                photo = tk.PhotoImage(file=obj["path"])
                self.photo_refs[obj["id"]] = photo
                item = self.canvas.create_image(x1, y1, image=photo, anchor="nw")
            except (tk.TclError, OSError, KeyError):
                item = self.canvas.create_text(x1, y1, text="[Image unavailable]",
                                               fill="#FF6584", anchor="nw")
        else:
            continue
        obj["canvas_item"] = item

    if self.selected_id is not None:
        obj = self.find_obj(self.selected_id)
        if obj and obj.get("visible", True):
            x1, y1, x2, y2 = obj["x1"], obj["y1"], obj["x2"], obj["y2"]
            if obj["type"] == "text":
                bbox = self.canvas.bbox(obj.get("canvas_item"))
                if bbox:
                    x1, y1, x2, y2 = bbox
            elif obj["type"] == "image":
                x2, y2 = x1 + 120, y1 + 90
            self.canvas.create_rectangle(x1-4, y1-4, x2+4, y2+4,
                                         outline=ACCENT, dash=(4, 3), width=2,
                                         tags="selection")
            self.canvas.tag_raise("selection")
    self.refresh_layers()

def on_press(self, event):
    x, y = event.x, event.y
    self.drag_start = (x, y)
    if self.current_tool == "select":
        current = self.canvas.find_withtag("current")
        if not current:
            self.select_object(None)
            return
        item = current[-1]
        obj = None
        for candidate in self.objects:
            if candidate.get("canvas_item") == item:
                obj = candidate
                break
        if obj is None or item in self.grid_ids:
            self.select_object(None)
            return
        self.select_object(obj["id"])
        self.drag_item_id = obj["id"]
```

```
        return

    if self.current_tool == "text":
        text = simpledialog.askstring("Add Text", "Enter your text:", parent=self.root)
        if text:
            self.snapshot()
            new_id = self.new_object("text", x, y, fill=self.current_color, text=text,
                                     font=self.font_family_var.get(),
                                     size=int(self.font_size_var.get()),
                                     bold=self.bold_var.get(), italic=self.italic_var.get())
            self.select_object(new_id)
            self.redraw()
            self.status("Text added")
        return

    self.snapshot()
    kind = self.current_tool
    new_id = self.new_object(kind, x, y, x, y, fill=self.current_color,
                             outline=self.current_color,
                             width=int(self.stroke_var.get()))
    self.temp_obj = new_id

def on_motion(self, event):
    if self.current_tool == "select" and self.drag_item_id is not None and self.drag_start:
        obj = self.find_obj(self.drag_item_id)
        if obj:
            dx = event.x - self.drag_start[0]
            dy = event.y - self.drag_start[1]
            obj["x1"] += dx
            obj["y1"] += dy
            obj["x2"] += dx
            obj["y2"] += dy
            self.drag_start = (event.x, event.y)
            self.redraw()
        return
    if self.temp_obj is not None and self.drag_start:
        obj = self.find_obj(self.temp_obj)
        if obj:
            obj["x2"], obj["y2"] = event.x, event.y
            self.redraw()

def on_release(self, event):
    if self.temp_obj is not None:
        self.select_object(self.temp_obj)
        self.temp_obj = None
        self.snapshot()
        self.status("Shape created")
    self.drag_item_id = None
    self.drag_start = None

def select_object(self, obj_id):
    self.selected_id = obj_id
    obj = self.find_obj(obj_id) if obj_id is not None else None
    if obj:
        self.current_color = obj.get("fill", ACCENT)
        if obj["type"] == "text":
            self.font_family_var.set(obj.get("font", "Arial"))
            self.font_size_var.set(str(obj.get("size", 32)))
            self.bold_var.set(obj.get("bold", False))
            self.italic_var.set(obj.get("italic", False))
        self.stroke_var.set(str(obj.get("width", 1)))
        self.status(f"Selected {obj['type']} • Layer {obj['id']}")
    else:
        self.status("Nothing selected")
    self.redraw()

def refresh_layers(self):
    if not hasattr(self, "layer_list"):
        return
    self.layer_list.delete(0, tk.END)
    # topmost objects appear at the top of the list
    for obj in reversed(self.objects):
        icon = "●" if obj.get("visible", True) else "○"
        name = obj["type"].title()
        if obj["type"] == "text":
            name += ": " + obj.get("text", "")[:16]
        elif obj["type"] == "image":
            name += ": " + os.path.basename(obj.get("path", "image"))
        self.layer_list.insert(tk.END, f"{icon}  {obj['id']} — {name}")
```

```
        if self.selected_id is not None:
            for i, obj in enumerate(reversed(self.objects)):
                if obj["id"] == self.selected_id:
                    self.layer_list.selection_set(i)
                    self.layer_list.see(i)
                    break

    def on_layer_select(self, event=None):
        selection = self.layer_list.curselection()
        if not selection:
            return
        reversed_objects = list(reversed(self.objects))
        index = selection[0]
        if index < len(reversed_objects):
            self.select_object(reversed_objects[index]["id"])

    def set_color(self, color):
        self.current_color = color
        self.apply_properties() if self.selected_id is not None else self.status("Color selected: " +
                color)

    def choose_color(self):
        result = colorchooser.askcolor(color=self.current_color, parent=self.root, title="Choose color")
        if result and result[1]:
            self.set_color(result[1])

    def apply_properties(self):
        obj = self.find_obj(self.selected_id)
        if not obj:
            self.status("Select an object first")
            return
        self.snapshot()
        obj["fill"] = self.current_color
        obj["outline"] = self.current_color
        obj["width"] = int(self.stroke_var.get())
        if obj["type"] == "text":
            obj["font"] = self.font_family_var.get()
            obj["size"] = int(self.font_size_var.get())
            obj["bold"] = self.bold_var.get()
            obj["italic"] = self.italic_var.get()
        self.redraw()
        self.status("Properties applied")

    def add_heading(self):
        self.snapshot()
        obj_id = self.new_object("text", 70, 70, fill=self.current_color,
                                 text="YOUR BIG IDEA", font=self.font_family_var.get(),
                                 size=44, bold=True)
        self.select_object(obj_id)
        self.set_tool("select")

    def add_subtitle(self):
        self.snapshot()
        obj_id = self.new_object("text", 74, 130, fill="#555555",
                                 text="A subtitle goes here", font="Arial",
                                 size=22, bold=False)
        self.select_object(obj_id)
        self.set_tool("select")

    def add_image(self):
        path = filedialog.askopenfilename(
            title="Choose an image",
            filetypes=[("Images", "*.png *.gif *.ppm *.pgm"),
                       ("All files", "*.*")]
        )
        if not path:
            return
        try:
            tk.PhotoImage(file=path)
        except tk.TclError:
            messagebox.showerror("Unsupported Image",
                                 "Tkinter supports PNG, GIF, PPM and PGM images on this setup.\n"
                                 "For JPG images, install Pillow or convert the image to PNG.")
            return
        self.snapshot()
        obj_id = self.new_object("image", 80, 80, path=path, fill="")
        self.select_object(obj_id)
        self.status("Image imported")
```

```
def delete_selected(self):
    if self.selected_id is None:
        self.status("Select an object to delete")
        return
    self.snapshot()
    self.objects = [o for o in self.objects if o["id"] != self.selected_id]
    self.selected_id = None
    self.redraw()
    self.status("Object deleted")

def duplicate_selected(self):
    obj = self.find_obj(self.selected_id)
    if not obj:
        self.status("Select an object to duplicate")
        return
    self.snapshot()
    clone = copy.deepcopy(obj)
    clone.pop("canvas_item", None)
    clone["id"] = self.next_id
    self.next_id += 1
    for key in ("x1", "x2"):
        clone[key] += 20
    for key in ("y1", "y2"):
        clone[key] += 20
    self.objects.append(clone)
    self.select_object(clone["id"])
    self.status("Object duplicated")

def bring_front(self):
    obj = self.find_obj(self.selected_id)
    if obj:
        self.snapshot()
        self.objects.remove(obj)
        self.objects.append(obj)
        self.redraw()
        self.status("Layer moved to front")

def send_back(self):
    obj = self.find_obj(self.selected_id)
    if obj:
        self.snapshot()
        self.objects.remove(obj)
        self.objects.insert(0, obj)
        self.redraw()
        self.status("Layer moved to back")

def toggle_visibility(self):
    obj = self.find_obj(self.selected_id)
    if obj:
        self.snapshot()
        obj["visible"] = not obj.get("visible", True)
        self.redraw()
        self.status("Layer visibility changed")

def change_background(self):
    result = colorchooser.askcolor(color=self.bg_color, parent=self.root, title="Canvas background")
    if result and result[1]:
        self.snapshot()
        self.bg_color = result[1]
        self.redraw()
        self.status("Background changed")

def toggle_grid(self):
    self.show_grid = not self.show_grid
    self.redraw()
    self.status("Grid " + ("enabled" if self.show_grid else "disabled"))

def set_zoom(self, zoom):
    self.zoom = zoom
    self.canvas.configure(width=int(self.canvas_w * zoom), height=int(self.canvas_h * zoom))
    self.canvas.scale("all", 0, 0, zoom, zoom)
    self.status(f"Zoom: {int(zoom * 100)}% (canvas view)")

def load_template(self, template):
    if self.objects and not messagebox.askyesno("Load Template", "Replace current design with this
            template?"):
        return
    self.snapshot()
    self.objects = []
```

```
        self.selected_id = None
        self.bg_color = "#FFFFFF"
        if template == "instagram":
            self.bg_color = "#F4E9E1"
            self.new_object("rect", 0, 0, 800, 520, fill="#F4E9E1", outline="#F4E9E1", width=0)
            self.new_object("text", 55, 60, fill="#242038", text="SUNDAY", font="Arial", size=54,
                    bold=True)
            self.new_object("text", 58, 125, fill="#7C5CFC", text="RESET", font="Arial", size=64,
                    bold=True)
            self.new_object("rect", 60, 225, 340, 8, fill="#FF6584", outline="#FF6584", width=0)
            self.new_object("text", 60, 260, fill="#242038", text="Slow down. Breathe.\nMake space for
                    you.", size=25, bold=False)
            self.new_object("oval", 520, 100, 730, 310, fill="#FFB7A5", outline="#FFB7A5", width=0)
            self.new_object("oval", 575, 150, 675, 250, fill="#7C5CFC", outline="#7C5CFC", width=0)
            self.new_object("text", 60, 440, fill="#242038", text="@YOURBRAND  •  SELF CARE CLUB",
                    size=15, bold=True)
        elif template == "story":
            self.bg_color = "#171923"
            self.new_object("rect", 0, 0, 800, 520, fill="#171923", outline="#171923", width=0)
            self.new_object("oval", 500, -30, 780, 250, fill="#7C5CFC", outline="#7C5CFC", width=0)
            self.new_object("text", 55, 55, fill="#52D6A6", text="YOU'RE INVITED", size=20, bold=True)
            self.new_object("text", 55, 125, fill="#FFFFFF", text="CREATIVE\nNIGHT", size=62, bold=True)
            self.new_object("text", 60, 300, fill="#D4D6E3", text="Music • Art • Community", size=23)
            self.new_object("rect", 60, 380, 235, 60, fill="#7C5CFC", outline="#7C5CFC", width=0)
            self.new_object("text", 88, 398, fill="#FFFFFF", text="SAVE YOUR SPOT", size=18, bold=True)
        elif template == "youtube":
            self.bg_color = "#161616"
            self.new_object("rect", 0, 0, 800, 520, fill="#161616", outline="#161616", width=0)
            self.new_object("oval", 520, 60, 760, 300, fill="#FF6584", outline="#FF6584", width=0)
            self.new_object("text", 45, 90, fill="#F7B267", text="THE ULTIMATE", size=24, bold=True)
            self.new_object("text", 45, 145, fill="#FFFFFF", text="DESIGN\nHACKS", size=66, bold=True)
            self.new_object("rect", 48, 330, 280, 62, fill="#7C5CFC", outline="#7C5CFC", width=0)
            self.new_object("text", 72, 348, fill="#FFFFFF", text="WATCH NOW", size=22, bold=True)
        elif template == "event":
            self.bg_color = "#F8F4EA"
            self.new_object("rect", 0, 0, 800, 520, fill="#F8F4EA", outline="#F8F4EA", width=0)
            self.new_object("rect", 0, 0, 20, 520, fill="#7C5CFC", outline="#7C5CFC", width=0)
            self.new_object("text", 65, 48, fill="#7C5CFC", text="CAMPUS • 2026", size=18, bold=True)
            self.new_object("text", 65, 110, fill="#25223A", text="DESIGN\nFESTIVAL", size=61,
                    bold=True)
            self.new_object("text", 70, 285, fill="#555166", text="Create. Connect. Inspire.", size=25)
            self.new_object("rect", 68, 360, 300, 68, fill="#25223A", outline="#25223A", width=0)
            self.new_object("text", 90, 382, fill="#FFFFFF", text="25 NOV  •  10 AM", size=19,
                    bold=True)
            self.new_object("oval", 520, 255, 720, 455, fill="#F7B267", outline="#F7B267", width=0)
            self.new_object("oval", 570, 305, 670, 405, fill="#FF6584", outline="#FF6584", width=0)
        else:
            self.bg_color = "#E9F2FF"
            self.new_object("rect", 0, 0, 800, 520, fill="#E9F2FF", outline="#E9F2FF", width=0)
            self.new_object("text", 65, 80, fill="#2C3A55", text="“Make it simple,", size=40, bold=True)
            self.new_object("text", 65, 145, fill="#7C5CFC", text="but significant.”", size=40,
                    bold=True)
            self.new_object("rect", 68, 240, 140, 6, fill="#FF6584", outline="#FF6584", width=0)
            self.new_object("text", 70, 285, fill="#58657A", text="— A NOTE TO CREATORS", size=16,
                    bold=True)
            self.new_object("oval", 620, 335, 740, 455, fill="#52D6A6", outline="#52D6A6", width=0)
        self.redraw()
        self.project_name = template.title() + " Template"
        self.title_label.configure(text=self.project_name)
        self.snapshot()
        self.status("Template loaded: " + template.title())

    def snapshot(self):
        state = {
            "objects": copy.deepcopy(self.objects),
            "bg_color": self.bg_color,
            "selected_id": self.selected_id,
            "project_name": self.project_name
        }
        # Remove runtime-only canvas IDs
        for obj in state["objects"]:
            obj.pop("canvas_item", None)
        if not self.history or self.history[-1] != state:
            self.history.append(state)
            if len(self.history) > 60:
                self.history.pop(0)
        self.future.clear()

    def restore_state(self, state):
```

```
        self.objects = copy.deepcopy(state["objects"])
        self.bg_color = state["bg_color"]
        self.selected_id = state["selected_id"]
        self.project_name = state["project_name"]
        self.title_label.configure(text=self.project_name)
        self.next_id = max([o["id"] for o in self.objects], default=0) + 1
        self.redraw()

    def undo(self):
        if len(self.history) < 2:
            self.status("Nothing to undo")
            return
        current = self.history.pop()
        self.future.append(current)
        self.restore_state(self.history[-1])
        self.status("Undo complete")

    def redo(self):
        if not self.future:
            self.status("Nothing to redo")
            return
        state = self.future.pop()
        self.history.append(copy.deepcopy(state))
        self.restore_state(state)
        self.status("Redo complete")

    def serialize(self):
        data = {
            "app": "PixelCraft Studio Plus",
            "name": self.project_name,
            "background": self.bg_color,
            "canvas_size": [self.canvas_w, self.canvas_h],
            "objects": []
        }
        for obj in self.objects:
            clean = {k: v for k, v in obj.items() if k != "canvas_item"}
            data["objects"].append(clean)
        return data

    def save_project(self):
        path = self.project_path
        if not path:
            path = filedialog.asksaveasfilename(
                title="Save PixelCraft Project", defaultextension=".json",
                filetypes=[("PixelCraft project", "*.json")]
            )
        if not path:
            return
        try:
            with open(path, "w", encoding="utf-8") as f:
                json.dump(self.serialize(), f, indent=2)
            self.project_path = path
            self.project_name = os.path.splitext(os.path.basename(path))[0]
            self.title_label.configure(text=self.project_name)
            self.status("Project saved")
        except (OSError, TypeError, ValueError) as exc:
            messagebox.showerror("Save Error", str(exc))

    def open_project(self):
        path = filedialog.askopenfilename(
            title="Open PixelCraft Project",
            filetypes=[("PixelCraft project", "*.json"), ("All files", "*.*")]
        )
        if not path:
            return
        try:
            with open(path, "r", encoding="utf-8") as f:
                data = json.load(f)
            self.objects = data.get("objects", [])
            for obj in self.objects:
                obj.setdefault("visible", True)
                obj.pop("canvas_item", None)
            self.bg_color = data.get("background", "#FFFFFF")
            self.project_name = data.get("name", os.path.basename(path))
            self.project_path = path
            self.selected_id = None
            self.next_id = max([o.get("id", 0) for o in self.objects], default=0) + 1
            self.title_label.configure(text=self.project_name)
            self.history.clear()
```

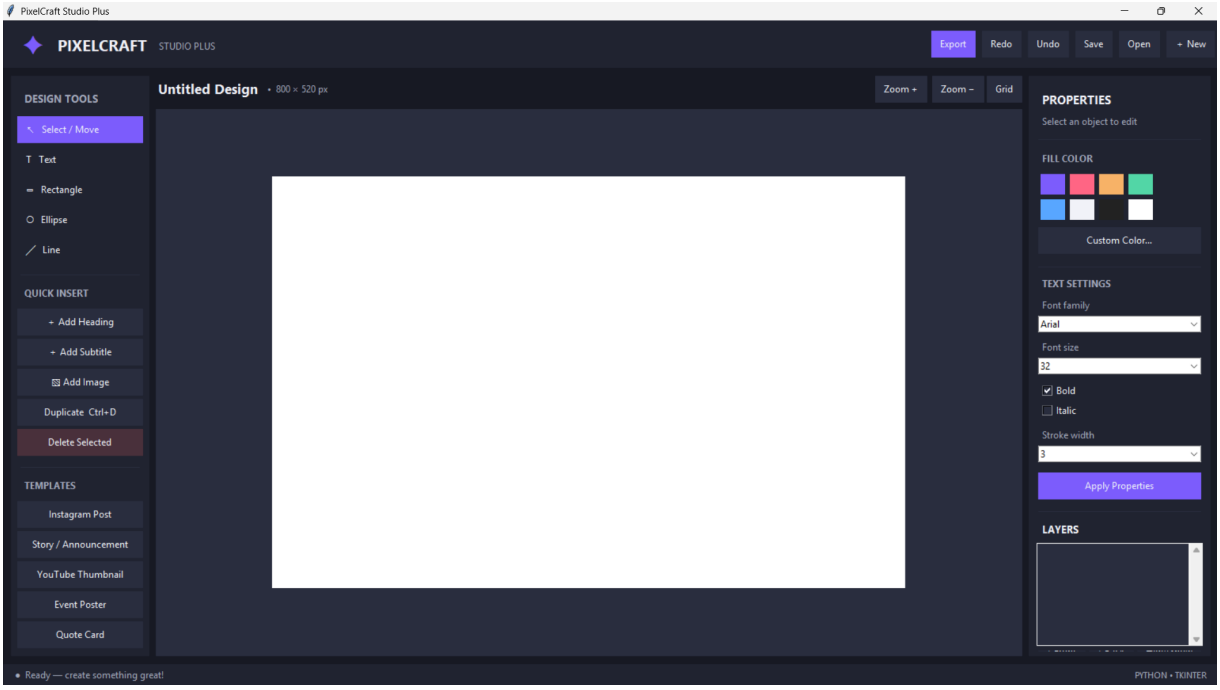
```
            self.future.clear()
            self.snapshot()
            self.redraw()
            self.status("Project opened")
        except (OSError, json.JSONDecodeError, TypeError, ValueError) as exc:
            messagebox.showerror("Open Error", str(exc))

    def new_project(self):
        if self.objects and not messagebox.askyesno("New Project", "Start a blank project? Save your
                work first if needed."):
            return
        self.snapshot()
        self.objects = []
        self.selected_id = None
        self.bg_color = "#FFFFFF"
        self.project_name = "Untitled Design"
        self.project_path = None
        self.title_label.configure(text=self.project_name)
        self.redraw()
        self.snapshot()
        self.status("New project")

    def export_design(self):
        path = filedialog.asksaveasfilename(
            title="Export Design", defaultextension=".ps",
            filetypes=[("PostScript", "*.ps"), ("EPS", "*.eps")]
        )
        if not path:
            return
        try:
            self.canvas.postscript(file=path, colormode="color",
                                   x=0, y=0, width=self.canvas_w, height=self.canvas_h)
            messagebox.showinfo("Export Complete",
                                "Design exported as PostScript.\n\n" + path +
                                "\n\nFor PNG export, install Pillow (optional).")
            self.status("Design exported")
        except (OSError, tk.TclError) as exc:
            messagebox.showerror("Export Error", str(exc))


if __name__ == "__main__":
    root = tk.Tk()
    app = PixelCraft(root)
    root.mainloop()
```

output:




RESULT:

Thus, an interactive 2D graphic design editor, PixelCraft Studio Plus, was developed using Python Tkinter with a modern dark interface, drawing tools for text, rectangles, ellipses, lines and images, colour and font customisation, layer management, ready-made templates, undo and redo, zoom and grid options, project save and open using JSON, and design export. The program demonstrates event-driven programming, GUI design, object-oriented programming, canvas drawing, file handling, exception handling and basic design-software development concepts.